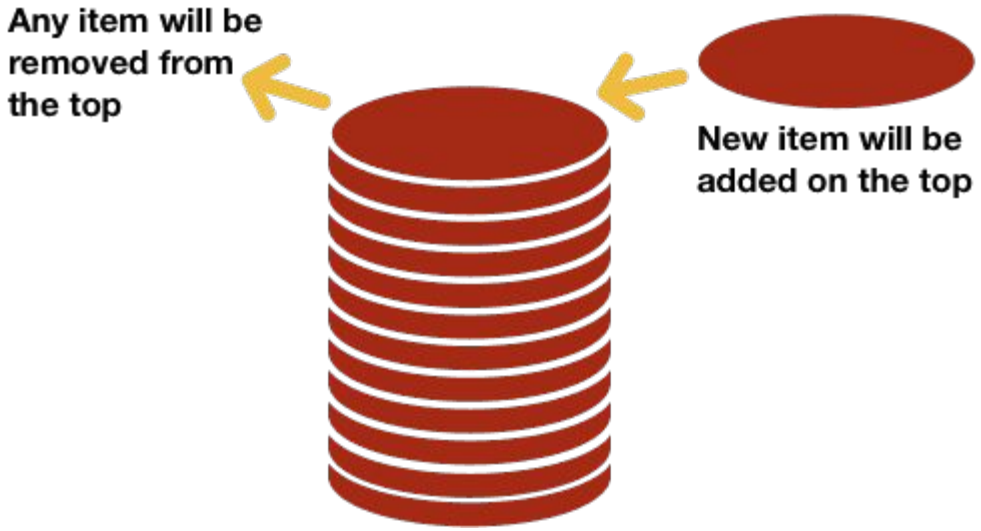
Stacks

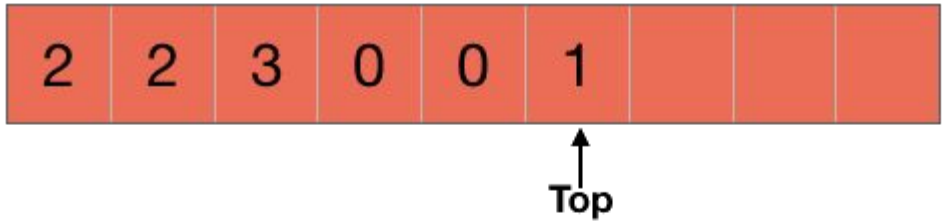
Lets start with a real-life example

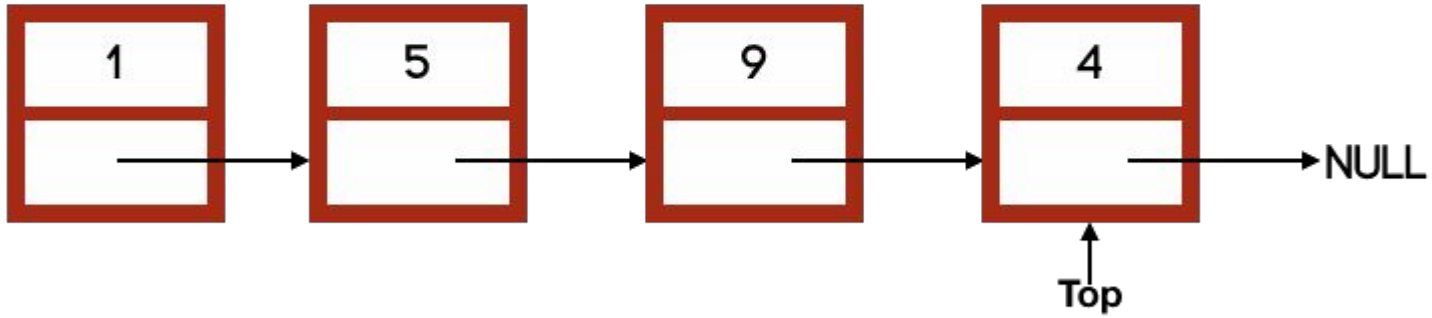
- In our day to day life, we see stacks of plates, coins, etc. All these stacks have one thing in common that a new item is added at the top of the stack and any item is also removed from the top i.e., the most recently added item is removed first.



Stack data structure

- Stack data structure follows the same kind of rules i.e., the most recently added item is removed first. It works on **LIFO (Last In First Out)** policy. It means that the item which enters at last is removed first.





New item will be added
and removed from top

Stack operations

- A stack supports few basic operations and we need to implement all these operations (either with a linked list or an array) to make a stack.
- These operations are:
  - Push
  - Pop
  - Top

Push

- he push operation adds a new element to the stack. As stated above, any element added to the stack goes at the top, so push adds an element at the top of a stack



Pop

- The pop operation removes and also returns the top-most (or most recent element) from the stack.



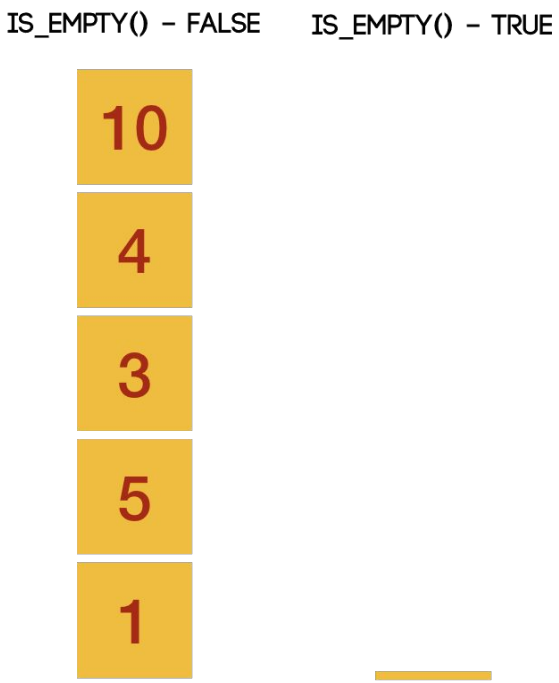
Top

- The Top operations only returns (doesn't remove) the top-most element of a stack.

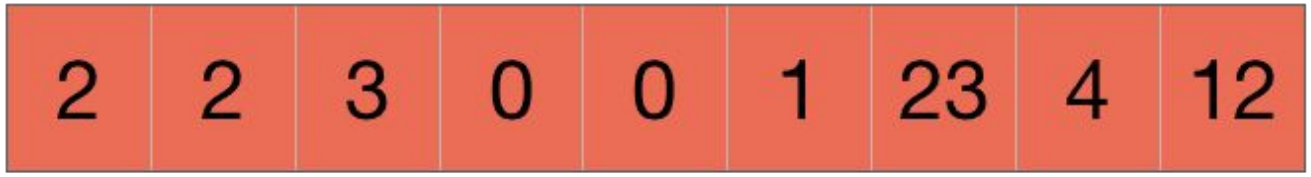


IsEmpty

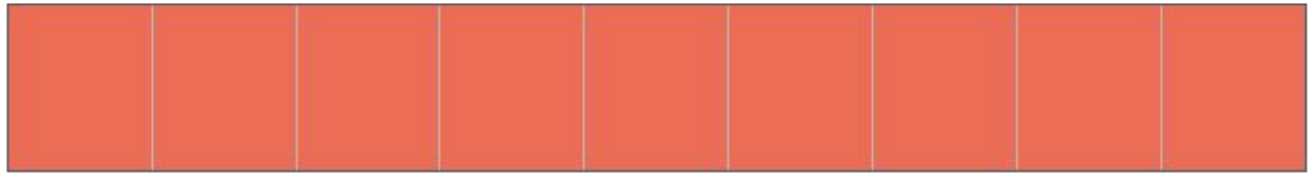
- This operation checks whether a stack is empty or not i.e., if there is any element present in the stack or not.



Handling errors with a stack.



PUSH(10) – STACK OVERFLOW



POP() – STACK UNDERFLOW

Stack - Abstract Data Type or Data Structure?

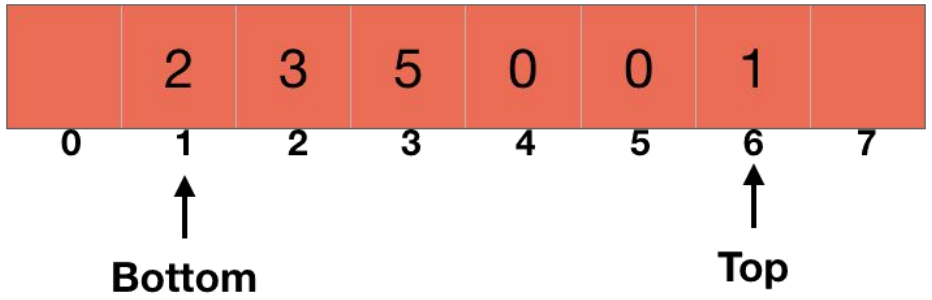
- **Abstract Data Type (ADT):** In the realm of computer science, a stack is often described as an abstract data type. An abstract data type defines a set of operations that can be performed on the data, without specifying how these operations should be implemented. For a stack ADT, the fundamental operations include:
  - Push: Adds an element to the top of the stack.
  - Pop: Removes and returns the element from the top of the stack.
  - Peek (or Top): Returns the element at the top of the stack without removing it.
  - IsEmpty: Checks if the stack is empty.
- The implementation details of how these operations are carried out can vary.
- **Data Structure:** On the other hand, a stack can also be considered a data structure. In this context, a stack is a specific way of organizing and storing data in memory. It follows the Last In, First Out (LIFO) principle, meaning that the last element added is the first one to be removed.
- The implementation of a stack as a data structure can be done using arrays or linked lists, for example. The choice of implementation details may vary based on the specific requirements of the application.

Applications of Stack

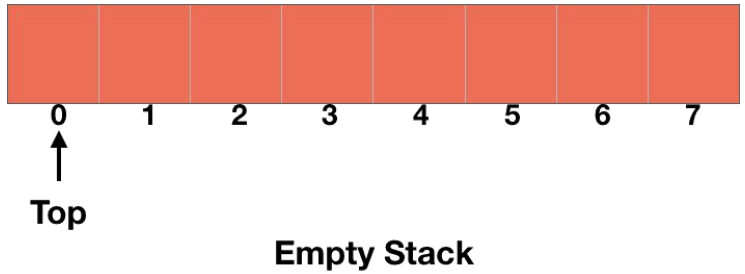
- There are many applications of a stack. Some of them are:
  - Stacks are used in backtracking algorithms.
  - They are also used to implement undo/redo functionality in a software.
  - Stacks are also used in syntax parsing for many compilers.
  - Stacks are also used to check proper opening and closing of parenthesis.

Stack Using Array

- We are going to use the element at the index 1 of the array as the bottom of the stack and the last element of the stack as the top of the stack as described in the picture given below.



- Since we need to add and remove elements from the top of the stack, we are going to use a pointer which is always going to point the topmost element of the stack. It will point to the element at index 0 when the stack is empty.
-

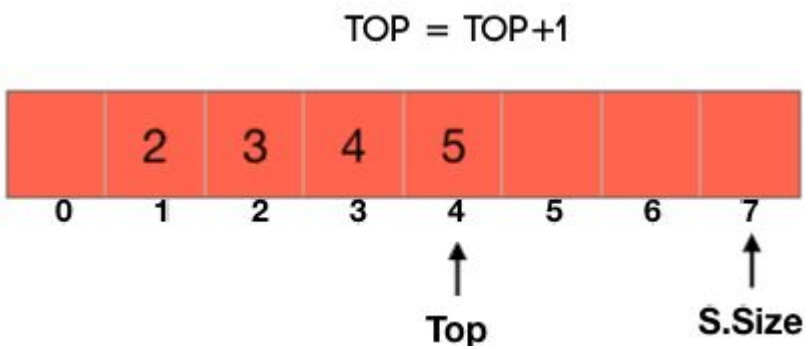


IsEmpty

```
IS_EMPTY(S)
   if S.top == 0
        return TRUE
   return FALSE
```

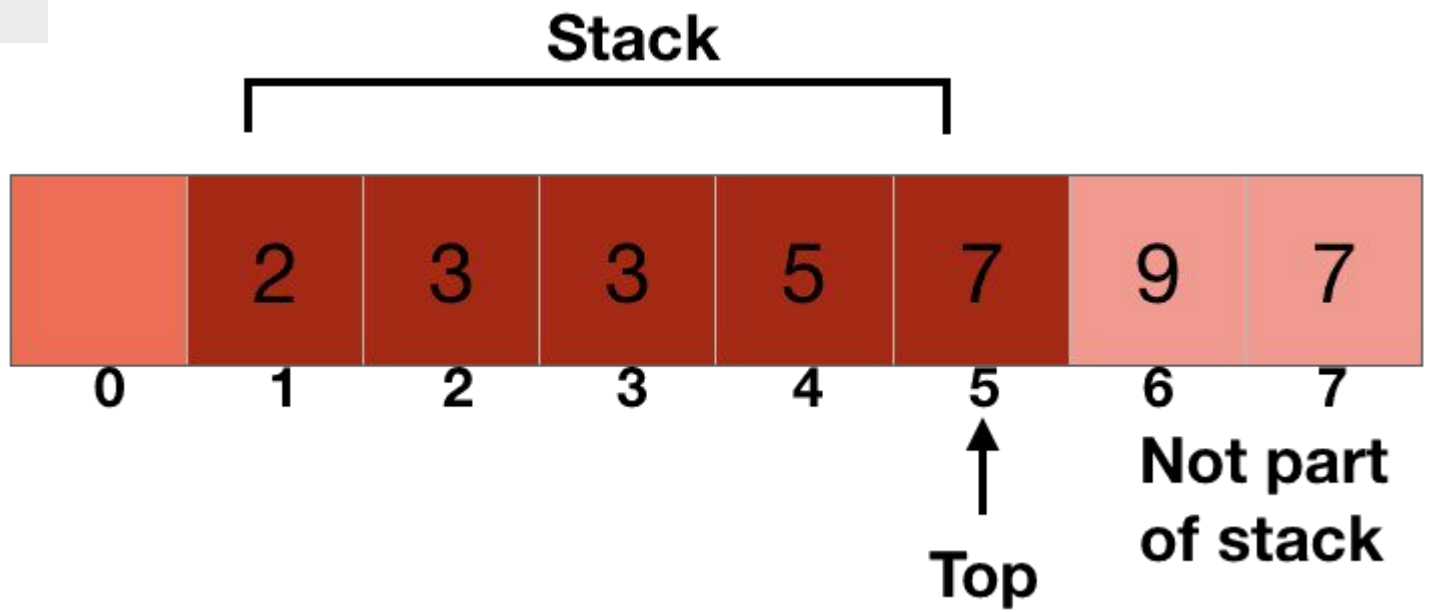
Push

```
PUSH(S, x)
   S.top = S.top+1
   if S.top > S.size
        Error "Stack Overflow"
   else
        S[S.top] = x
```



Pop

```
POP(S)
   if IS_EMPTY(S)
        Error “Stack Underflow”
   else
        S.top = S.top-1
        return S[S.top+1]
```



C language Implementation

```c
int pop() {
if(is_empty()) {
   printf("Stack Underflow\n");
   return -1000;
}
else {
   top = top-1;
   return S[top+1];
}
}
```

```c
#include <stdio.h>

#define SIZE 10

int S[SIZE+1];
int top = 0;

int is_empty() {
if(top == 0)
   return 1;
return 0;
}

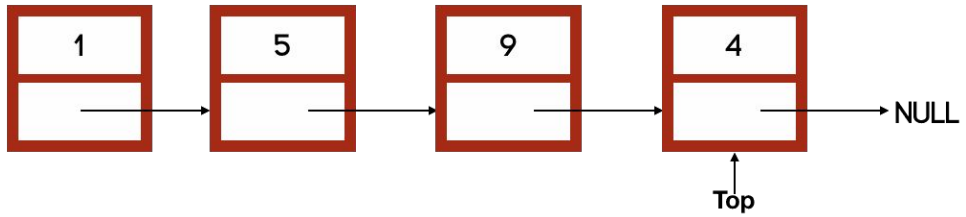
void push(int x) {
top = top+1;
if(top > SIZE) {
   printf("Stack Overflow\n");
}
else {
   S[top] = x;
}
}
```

```c
int main() {
pop();
push(10);
push(20);
push(30);
push(40);
push(50);
push(60);
push(70);
push(80);
push(90);
push(100);
push(110);

int i;
for(i=1; i<=SIZE; i++) {
   printf("%d\n",S[i]);
}
return 0;
}
```

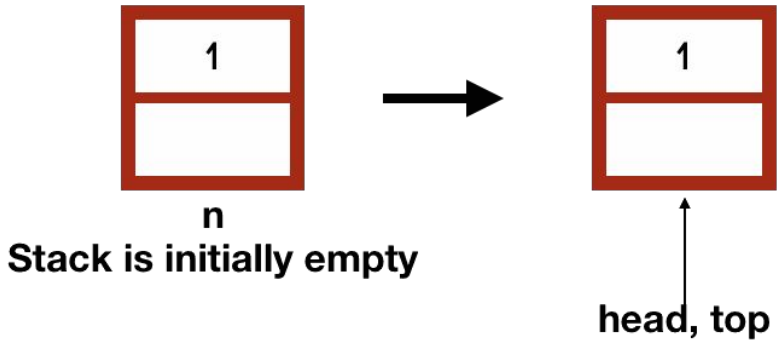
Stack Using Linked List

- A stack using a linked list is just a simple linked list with just restrictions that any element will be added and removed using push and pop respectively.



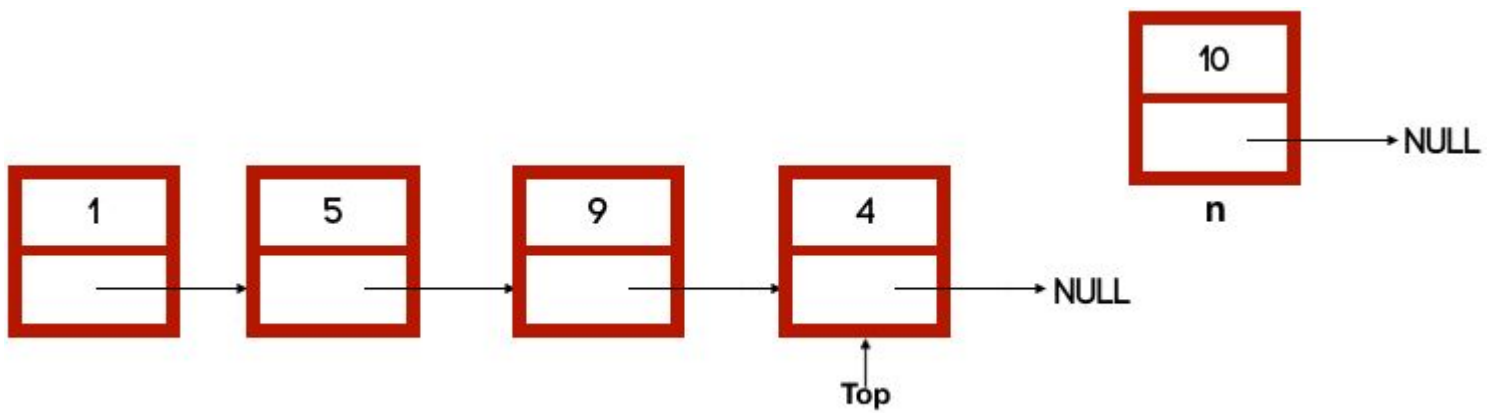
IsEmpty

```
IS_EMPTY(S)
   if S.top == null
        return TRUE
   return FALSE
```



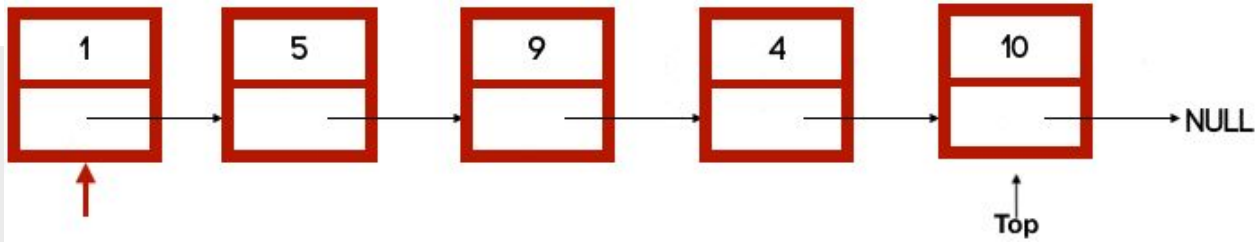
Push

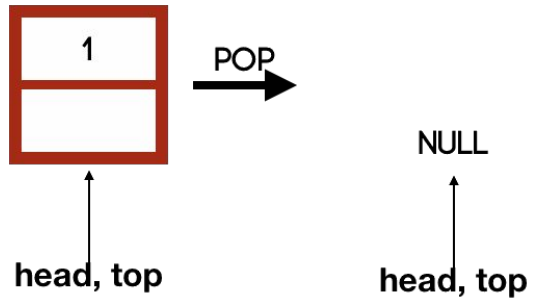
```
PUSH(S, n)
   if IS_EMPTY(S) //stack is empty
        S.head = n //new node is the head of the linked list
        S.top = n //new node is the also the top
   else
        S.top.next = n
        S.top = n
```



Pop

```
POP(S)
   if IS_EMPTY(S)
        Error "Stack Underflow"
   else
        x = S.top.data
        if S.top == S.head //only one node
             S.top = NULL
             S.head = NULL
        else
             tmp = S.head
             while tmp.next != S.top //iterating to the node previous to top
                  tmp = tmp.next
             tmp.next = NULL //making the next of the node null
             S.top = tmp //changing the top pointer
        return x
```





If the stack has only one node (*top* and *head* are same), we will just make
both *top* and *head* null.

References

- https://www.codesdope.com/course/data-structures-introduction/
- Mark Allen Weiss, "Data Structures and Algorithm Analysis in C++", Third Edition, Pearson Education, 2006
- Data Structures by D. Samantha et al.
- Various online sources